

VMs on a laptop or server can quickly eat up resources.

Containers solve many of the problems VMs introduce. Think of a container like a lightweight box that holds your app and everything it needs to run—but without the baggage of an entire operating system. It shares the host system's OS kernel, making it faster and more efficient.

Containers start up in seconds, take up less space, and are much easier to move between environments. For example, you can build a container on your laptop, push it to a cloud service, and be confident it will run the same way everywhere.

| Feature        | Virtual machines                | Containers                           |
|----------------|---------------------------------|--------------------------------------|
| OS requirement | Full guest OS per VM            | Share host OS kernel                 |
| Startup time   | Minutes                         | Seconds                              |
| Resource usage | Heavy                           | Lightweight                          |
| Portability    | Moderate                        | Very high                            |
| Use case fit   | Legacy apps, OS-level isolation | Modern app deployment, microservices |

Table 1: The differences between containers and VMs

So how does one choose between VMs and containers? Well, you can use VMs when you need to run different operating systems or need strong isolation (e.g., for security or regulatory reasons), and use containers when you want speed, simplicity, and portability—especially for modern applications that need to scale.

In short, containers don't replace VMs completely, but for many use cases they offer a smarter, faster, and more developer-friendly way to run applications.

## Getting started with Docker: Core concepts and workflow

So now that we understand why containers are such a big deal, the next question is: how do you actually use them? That's where Docker comes in.

Docker is the most popular platform for building, running, and managing containers. It takes all the complexity of container technology and wraps it in a clean, developer-friendly toolset. Whether you're working on a small web app or a large microservices project, Docker helps you package your application and run it anywhere with ease.

### Key concepts in Docker

Let's break down a few basic but important terms you'll see when using Docker.

**Image:** Think of an image as a recipe. It defines everything your app needs—like the code, system libraries, dependencies, and runtime.

**Container:** A container is a running instance of an image. If the image is the recipe, the container is the actual dish made from it.

**Dockerfile:** This is a text file that contains instructions for building a Docker image. It might say: "Use Python 3.10, install these libraries, copy this code."

**Docker Hub:** This is a public registry where you can find pre-made images (like Node.js, PostgreSQL, or Ubuntu) or upload your own.

**Volumes:** These allow you to store data outside your container so that it's not lost when the container shuts down.

### Containerising a simple app

Let's say you have a basic Python app that prints "Hello, world." With Docker, you could:

- Write a Dockerfile that tells Docker to use Python and run your script.
- Build an image using the command `docker build -t hello-world`.
- Run the container with `docker run hello-world`.

Boom! Your app is now running in an isolated, portable container that you could ship to another computer or deploy to the cloud.

### What is Docker Compose?

Most real-world applications need more than one service. For example, your web app may need:

- A backend server
- A frontend UI
- A database

Docker Compose is a tool that lets you define and run multi-container apps using a simple `docker-compose.yml` file. It's perfect for local development and testing complex setups without needing a full cloud environment.

Docker has made it incredibly easy to move from "it works on my machine" to "it works everywhere."

By now, you're probably excited about how easy Docker makes working with containers. But as with any powerful tool, it's important to talk about security, and also recognise that Docker isn't the only player in town.

### How secure are containers?

Containers are generally more secure than running everything directly on your host machine, but they're not foolproof. Here are a few key things to keep in mind.

**Isolation isn't absolute:** Unlike virtual machines, containers share the host's operating system kernel. If a container breaks out, it could potentially affect the host or other containers.